Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Siyi Luo (team leader and only member; individual project)

Code and data: `https://github.com/borispie/who-wrote-it-llm-id`

1 Problem definition

Chatbots now write a lot of the text we read. In this project I ask a simple question: *if I give a computer an answer written by a large language model (LLM), can it tell which LLM wrote it?*

I set this up as a classification problem with six classes. Each example has three parts: the name of the LLM, the user's prompt (the *input*) and the LLM's answer (the *output*). The label is the LLM family: GPT, Claude, Gemini, Llama, Qwen or DeepSeek. Random guessing would be right 1 time in 6, which is 16.7%.

In this report, "LLM" means the chatbot that wrote an answer, and "classifier" means my own model. I call the habits that make one LLM's answers easy to recognize its *fingerprint*. I trained the two required classifiers, a CNN and an RNN (a bidirectional LSTM, or "BiLSTM"), and used them to answer four questions:

- **RQ1.** Can we tell which LLM wrote an answer, using only the answer?
- **RQ2.** Does it help to also show the classifier the prompt?
- **RQ3 (extra credit).** Does the fingerprint still work on a kind of task the classifier never saw?
- **RQ4 (extra credit).** What makes the LLMs different, and what happens if we remove those differences?

2 Dataset curation

Where the data comes from. I did not collect any chats myself. I used the public answers released with **AlpacaEval** [1], a well-known benchmark for chatbots (GitHub `tatsu-lab/alpaca_eval`, Apache-2.0 license). For hundreds of models, it stores the model's answers to the same 805 prompts, and each record already has the three fields I need: model name, prompt and answer. I picked one model per family (Table 1). I chose this source for three reasons:

1. It is public and has no private or personal data.
2. Every model answered the *same* prompts. So the classes are perfectly balanced, and the topic of a prompt cannot give away which LLM answered it (this matters for RQ2).
3. It has all six families I wanted.

My script `src/build_dataset.py` downloads the files from one fixed version of the repository (commit `cd543a1`), so anyone can rebuild exactly the same dataset.

Cleaning. One prompt had an empty answer from one model, so I removed that prompt for all six models. That leaves 804 prompts $\times$ 6 = 4,824 answers, exactly 804 per family. They are saved in

Table 1. The six LLMs (one model per family). Every model answered the same 804 prompts.

Family	Model (name in AlpacaEval)	Answers	Median words	Mean words
GPT	`gpt-4o-2024-05-13`	804	291	286
Claude	`claude-3-5-sonnet-20240620`	804	200	229
Gemini	`gemini-pro`	804	233	229
Llama	`Meta-Llama-3-70B-Instruct`	804	312.5	299
Qwen	`Qwen2-72B-Instruct`	804	254	260
DeepSeek	`deepseek-llm-67b-chat`	804	181	189

`data/llm_responses.csv` with the columns `llm_name`, `llm_input` and `llm_output`, plus a prompt id, the family, the prompt's source, the task type and the split. I kept every answer as it was written, including markdown (`#`, `**`, bullet points), line breaks and emojis, because how an LLM formats its answer is part of its style.

Task types (needed for RQ3). AlpacaEval does not say what kind of task each prompt is, so I labeled the prompts with simple keyword rules. For example, "python" or "SQL" means *coding*, "solve" means *math*, and "poem" or "email" means *writing*; everything else is *QA / advice*. Rules make mistakes ("Write a script for a YouTube video" is writing, not coding), so I checked them by hand and fixed 38 labels (30 wrong coding/math labels and 8 prompts the rules missed). The final counts are 549 QA / advice, 162 writing, 60 coding and 33 math prompts.

Train / validation / test split. I split the data **by prompt**: 70% of the prompts for training (562), 15% for validation (120) and 15% for testing (122). All six answers to one prompt always go to the same part, so every test prompt is new to the classifier, just like in real use.

3 Models and training

From text to numbers. I wrote a small tokenizer that splits the text into words, numbers and punctuation marks. It also keeps formatting marks as their own tokens: a line break becomes `<nl>`, and markdown symbols like `**` and `#` are kept. Words are lowercased. Answers are long (median 232 words) and I only had a CPU, so I kept the **first 200 and last 50 tokens** of each answer. The vocabulary (14,732 tokens) is built from the training answers only. For RQ2, the prompt (up to 64 tokens; 128 when it is the only input) goes in front of the answer, with a `<sep>` token in between.

The two classifiers. Both start the same way: each token becomes a list of 128 numbers (an *embedding*) that the model learns during training. Table 2 lists all settings.

- **CNN** (following Kim [2]). The CNN slides 300 small filters over the text. Each filter looks at 3, 4 or 5 tokens at a time and learns to spot one short pattern, such as "here's a" or a line break followed by `**`. For each filter, the model keeps only its strongest match anywhere in the answer (*max-pooling*). A final layer turns these 300 numbers into 6 scores, one per family.
- **BiLSTM** [3]. An LSTM reads the tokens one at a time and keeps a memory of what it has read so far. A bidirectional LSTM uses two of them: one reads left to right, the other right to left. I take the average and the maximum of its outputs over the whole answer, and a final layer gives 6 scores.

Training. Both classifiers are trained the same way. After each pass over the training data (an *epoch*), I check the score on the validation set and keep the best version of the model. If the score does not improve

Table 2. Classifier and training settings.

	CNN	BiLSTM
Input	first 200 + last 50 tokens of the answer	
Embedding	128 numbers per token, learned from scratch (20% dropout)	
Main layer	300 filters: 100 each of width 3, 4, 5	1 BiLSTM layer, 64 units per direction
Summary of the answer	strongest match of each filter	average + maximum over all tokens
Output	50% dropout, then a linear layer with 6 outputs	
Parameters	2,041,402	1,986,566
Time per epoch (1 CPU thread)	23–37 seconds	138–179 seconds
Training (the same for both)		
Loss, optimizer	cross-entropy; AdamW [4] (learning rate 0.001, weight decay 0.0001)	
Batch size, epochs	32; at most 15 epochs; gradient clipping at 1.0	
Early stopping	keep the best epoch (by validation macro-F1); stop after 3 epochs with no gain	
Random seeds	CNN: 3 (RQ2 bias test: 5). BiLSTM: 3 for RQ1, 1 for the other experiments	

for 3 epochs in a row, training stops early. The test set is used only once, at the end. Results change a little with the random seed, so I trained each CNN setting 3 times and report the average $\pm$ standard deviation. The BiLSTM is about 5–6 times slower, so it got 3 seeds for RQ1 and 1 seed for the other experiments. Everything ran in PyTorch on a CPU with 2 cores and no GPU.

Baselines and metrics. To know whether the neural classifiers are any good, I compare them with random guessing (16.7%) and with a simple classic model, **TF-IDF + logistic regression**. It counts which words and word pairs appear in an answer and learns one weight for each. I report **accuracy** (the share of answers labeled correctly) and **macro-F1**, the average F1 score over the six families. A family's F1 is high only if the classifier finds most of that family's answers *and* is usually right when it guesses that family.

4 Results

4.1 RQ1: Can we tell which LLM wrote an answer?

Yes, much better than guessing. Using only the answer, the CNN picks the right LLM 63.0% of the time and the BiLSTM 59.6% of the time (Table 3). Guessing gets 16.7%, so both are about 3.5 to 4 times better.

The CNN is about 3 points ahead of the BiLSTM, but their results over the 3 seeds overlap (CNN 60.9–65.2%, BiLSTM 58.5–61.5%), so I cannot say the CNN is clearly better. The simple TF-IDF baseline (63.3%) does as well as the CNN. This suggests that most of the fingerprint is in *which* words, short phrases and formatting marks appear. Word counts, or a CNN with short filters, can catch that easily. Reading the answer in order, like the BiLSTM does, does not add much here.

Table 3. RQ1 test results using only the answer. CNN and BiLSTM: average $\pm$ standard deviation over 3 seeds. The last six columns are the F1 score (%) for each family.

Classifier	Accuracy (%)	Macro-F1 (%)	GPT	Claude	Gemini	Llama	Qwen	DeepSeek
CNN	63.0 ± 2.1	61.9 ± 2.6	58.6	79.6	68.8	77.0	35.5	51.7
BiLSTM	59.6 ± 1.6	58.8 ± 2.2	54.9	73.2	67.4	73.6	29.8	53.7
TF-IDF + LR (baseline)	63.3	62.1	58.8	77.9	72.6	76.0	33.2	54.2
Chance	16.7	–	–	–	–	–	–	–

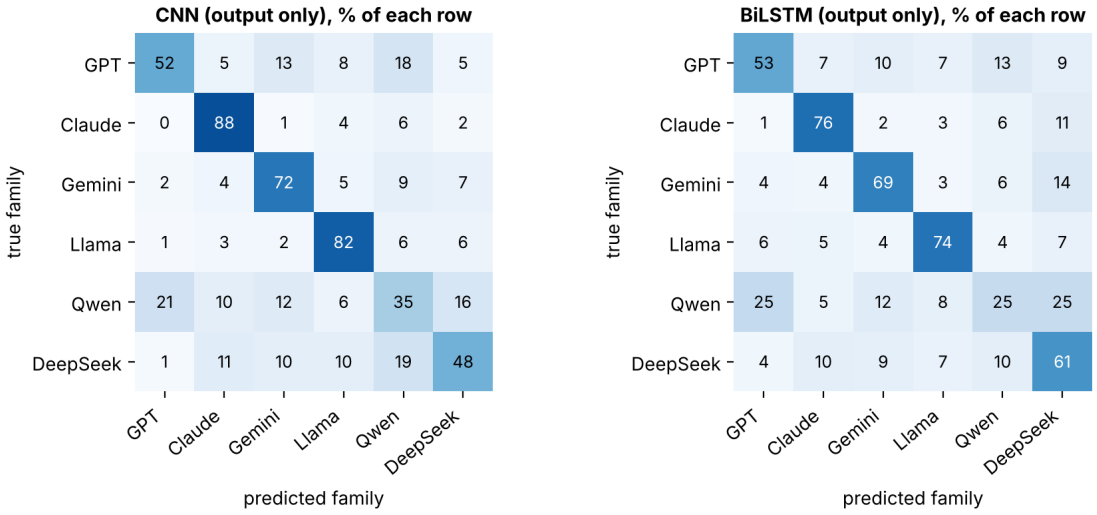


Figure 1. Confusion matrices on the test set (3 seeds together). Each row is the true LLM and shows where its answers went (% of the row). The diagonal is the share of that LLM's answers that were labeled correctly.

Some LLMs are much easier to spot than others (Figure 1). Claude and Llama are the easiest: the CNN finds 88.3% of Claude's answers and 82.2% of Llama's. Qwen is the hardest (35.0%) and is often mistaken for GPT or DeepSeek. RQ4 gives a hint why: Qwen shares two of GPT's rarer habits (markdown headers and the curly apostrophe), but it uses bold and bullet lists much less than GPT, so its style sits between GPT's and DeepSeek's.

Short answers are much harder. For answers under 100 words, the CNN is right only 38.2% of the time, compared with 70.3% for answers of 300 words or more. A short answer simply has fewer clues.

Two smaller findings. The best validation score came after about 11 epochs for the CNN and 13 for the BiLSTM (average of 3 seeds). All three BiLSTM runs and one CNN run hit the 15-epoch limit, but in each one the last epoch already scored below the best one, so the limit probably cost little. And a majority vote of all 7 trained classifiers (3 CNNs, 3 BiLSTMs and TF-IDF) gets 67.6%, a bit better than the best single run (65.2%), because the classifiers make partly different mistakes.

4.2 RQ2: Does the prompt help?

Table 4 compares three inputs: the prompt only, the answer only, and both.

Table 4. RQ2 test accuracy (%). CNN: average of 3 seeds. BiLSTM: 3 seeds for "answer only", 1 seed for the other two rows.

What the classifier sees	CNN	BiLSTM	TF-IDF + LR
Prompt only (input)	16.7	16.7	16.7
Answer only (output)	63.0	59.6	63.3
Prompt + answer	64.7	58.6	63.3

(i) Can the prompt alone predict the LLM? No. With only the prompt, every classifier gets exactly 16.7%, the guessing rate. This is what my dataset design predicts. All six LLMs answered every prompt, so each prompt appears once with every label. The classifier has to give all six copies of a prompt the same answer,

and exactly one of the six is right.

(ii) Does adding the prompt help? No. Prompt + answer scores about the same as the answer alone (CNN 64.7% vs. 63.0%), and the difference is smaller than the change between seeds. The prompt is the same for all six LLMs, so it cannot tell them apart.

(iii) When could the prompt alone work surprisingly well? When the data is collected in a biased way. In many real datasets, each prompt is answered by only one LLM, and *which* LLM answers depends on where the prompt came from (for example, different websites or user groups use different chatbots). I simulated this with my data: I kept one answer per prompt and let the prompt's source decide the LLM (for example, every prompt from the Koala collection was answered by Gemini). Now a prompt-only classifier gets 57.4% (TF-IDF) to 60.0% (CNN), even though the prompts say nothing about the LLM. That is far above the 30.3% you get by always guessing the most common LLM. It is detecting where the prompt came from, not who wrote the answer. With a fair collection it stays near guessing (17.2–19.8%). So a prompt-only test is a cheap way to check a dataset for this kind of bias. (I repeated this random sampling 5 times and report the average; here the CNN used patience 5 and TF-IDF used single words only.)

5 In-depth analysis

5.1 RQ3: Does the fingerprint work on a new kind of task?

I ran the two tests suggested in the assignment:

- **Writing shift:** train on coding, math and QA / advice prompts; test on writing prompts.
- **Coding shift:** train on QA / advice prompts only; test on coding prompts.

A classifier that never saw writing might also do worse just because it had less training data. To make the comparison fair, I trained a **control** with the same amount of data that *did* include half of the new task's prompts, and tested both on the other half (then I swapped the halves). The gap between the control and the classifier that never saw the task is the real cost of the task being new.

Table 5. RQ3 test accuracy (%). *Own tasks*: new prompts from the tasks the classifier trained on. The control and the classifier that never saw the task are tested on the same target-task answers. Drop = control minus never-saw-it, in points, with a 95% range from resampling the test prompts. Number of seeds in parentheses.

Test	Classifier	Own tasks	Control	Never saw the task	Drop (95% range)
Writing shift	CNN (3)	66.1 ± 0.3	49.0 ± 0.4	46.6 ± 1.5	+2.4 [+0.9, +3.9]
Writing shift	BiLSTM (1)	61.5	43.1	40.8	+2.3 [−0.9, +5.2]
Coding shift	CNN (3)	67.0 ± 1.7	51.4 ± 1.1	48.6 ± 1.7	+2.8 [−0.2, +5.8]

(i) Does performance drop? Only a little. Never seeing writing costs the CNN about 2.4 points, and never seeing coding about 2.8 points (Table 5). To see whether these drops are real, I resampled the test prompts 4,000 times. The writing drop stays above zero (0.9 to 3.9 points), but the coding drop might be zero (-0.2 to 5.8 points). The task itself matters much more: the CNN gets about 66–67% on new prompts from its own tasks, but writing and coding answers are hard even for the control (about 49–51%).

(ii) Are some LLMs easier to spot on a new task? Yes. Claude and Llama stay the easiest. Without ever training on writing, the CNN still finds 62% of Claude's and 60% of Llama's writing answers. Their habits, like Claude's "Here's…" and Llama's "What a…!" openings, show up in every task type. GPT is the

opposite: on writing, it is found only about 31% of the time, compared with 51.9% on the whole test set. An email or a poem has no headers or bullet lists, so GPT loses its strongest clue.

(iii) A general fingerprint or task-specific patterns? Mostly general. The drop for a new task is small, so the classifier learned habits that carry over between tasks. But the fingerprint is weaker when the prompt asks for a fixed form (an email, a poem, some code), because then every LLM follows that form.

5.2 RQ4: What makes the LLMs different?

Style statistics. Figure 2 compares simple features over all 804 answers of each LLM. The biggest differences are in **formatting**:

- GPT uses markdown headers (#) in 30% of its answers. The others rarely do (Qwen 9%, the rest 1–2%).
- GPT and Llama use bold text in about 60% of their answers; Claude and DeepSeek almost never do.
- Claude likes numbered lists (74% of its answers).
- Llama and GPT write the longest answers, DeepSeek the shortest.
- Word variety (the share of different words in a 50-word window [5]) is about the same for every LLM (0.74–0.79), so it is a weak clue.

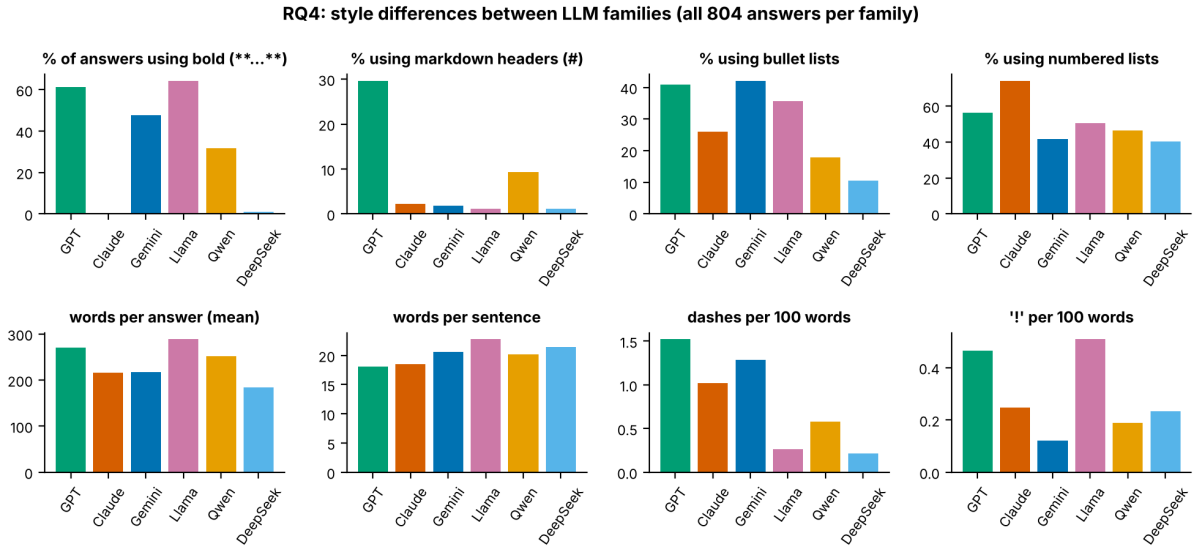


Figure 2. Style differences between the LLMs. Top row: % of answers that use each kind of formatting at least once. Bottom row: averages. Here a word is a run of letters, so the word counts are a little lower than in Table 1, which splits the text at spaces.

Some small habits work almost like a signature (Table 6). GPT uses the curly apostrophe (’) in 22% of its answers, and only Qwen also does it often (8%). Llama writes bullets with ∗ and is the only one that opens with “What a. . . !” (for example “What a great question!”). Claude often starts with “Here’s”, Llama with “Here is” or “Here are”, and GPT with “Sure” or “Certainly”.

What do the classifiers look at? The TF-IDF model’s strongest features are mostly formatting marks (∗∗, ###, a line break before a list) and typical phrases (“sure!”, “here’s a”, “what a”, “therefore”). I also looked

Table 6. Signature habits: % of each LLM's answers that show the habit.

Habit	GPT	Claude	Gemini	Llama	Qwen	DeepSeek
uses the curly apostrophe (')	22	0	0	0	8	0
writes bullets as *	0	0	17	35	1	8
writes bullets as -	41	25	25	0	17	2
opens with *Sure / Certainly / Absolutely / Of course*	27	9	0	0	12	10
opens with *Here's / Here is / Here are*	1	25	0	27	3	4
opens with *What a / What an*	0	0	0	15	0	0

at the pieces of text that switch on the CNN's strongest filters, and they match the same habits: "### step" headers for GPT, "here's why:" for Claude, "what a great question!" for Llama and "therefore," for Gemini. A logistic regression that sees only 22 simple style numbers gets 46.0%. The 7 formatting numbers alone give 39.2%, more than the 12 wording numbers, such as sentence length and punctuation (30.6%), or the 3 length numbers (28.3%).

Removing one kind of signal. Finally, I changed the answers in five ways (Table 7) and tested the CNN in two ways. First, the normal CNN is tested on the changed answers: this shows how much it *relies* on what I removed. Second, a new CNN is trained on the changed answers: this shows whether *enough other information is left* to recognize the LLM.

Table 7. RQ4: CNN test accuracy (%) after removing one kind of signal (average of 3 seeds).

What the CNN sees	Normal CNN, tested on changed answers	New CNN, trained on changed answers
The full answer (nothing removed)	63.0	63.0
No markdown and no line breaks	32.5	48.1
Words only (no punctuation or formatting)	27.4	46.6
Structure only (words replaced by <w>)	48.5	61.3
First 30 tokens only	43.5	46.7
A middle piece only (tokens 50–150)	45.4	50.8

(i) What does the classifier use? Mostly the structure of the answer. When the words are replaced by a placeholder, so the CNN mostly sees punctuation, line breaks and markdown, a new CNN still gets 61.3%, close to the 63.0% for the full answer. With only the words, it gets 46.6%.

(ii) Words or structure? Both, but structure is stronger. Words alone are still far above guessing, so word choice ("here's", "certainly", "therefore") carries real signal. The shape of the answer carries more.

(iii) Does removing these signals hurt? Yes, a lot. Without markdown and line breaks, the normal CNN drops from 63.0% to 32.5%. A new CNN trained without them gets some of it back (48.1%), but not all. The fingerprint is also spread over the whole answer: the first 30 tokens give 46.7%, and a piece from the middle gives 50.8%. So just asking an LLM to answer in plain text, without markdown, would already hide much of its identity.

6 Lessons and experience

1. The dataset design decides what you can conclude. Because all six LLMs answered the same prompts, the prompt-only test lands exactly on chance, and the topic of a prompt cannot sneak into my RQ1 numbers.

My simulation showed how a biased collection could create a fake fingerprint: 57.4% accuracy from prompts alone. Now, when I see a high accuracy, my first question is: what else could the model be using?

2. How you split the data matters. I split by prompt. Later I tried a random split by row: TF-IDF fell from 63.3% to 47.3%. With a row split, the other five answers to a test prompt sit in the training data under other labels, and their shared topic words point to the wrong LLM. The split changed my result by about 16 points.

3. Start with a simple baseline. TF-IDF + logistic regression trained in seconds and was as good as my neural classifiers. The BiLSTM was about 5–6 times slower and a bit worse. With only about 3,400 training answers, a deep model does not win automatically.

4. Preprocessing is a modeling choice. I kept line breaks and markdown as tokens because I guessed that formatting mattered, and RQ4 showed it is the strongest signal. The usual "lowercase and remove punctuation" step would have thrown much of it away.

5. Check labels made by rules. My keyword rules put "Write a script for a YouTube video" under coding. Checking the labels by hand found 38 mistakes, and RQ3 depends on these labels.

6. Plan around your computer, and report the spread. I had no GPU, so I timed one epoch before planning anything. My first BiLSTM needed about 5 minutes per epoch, so I made it smaller and cut the answers to 250 tokens. I also learned that one run is not enough: the same CNN got anywhere from 60.9% to 65.2% depending on the random seed.

7 Limitations

- **One model per family.** The classifier learns these six models, not whole families. The models are also from different years (Gemini Pro and DeepSeek-LLM-67B are from 2023, the others from 2024), so some differences may come from model age.
- **Default settings only.** All answers come from AlpacaEval's standard setup. A user who asks for "no markdown" could hide much of the fingerprint.
- **Small test set.** The test set has 732 answers, so differences of 2–3 points can be noise.
- **Not proof.** About 1 in 3 answers is still labeled wrong, so a tool like this must never be used as proof of who (or what) wrote a text.

How to reproduce. The code, the dataset, all results and the training logs are at `https://github.com/borispie/who-wrote-it-llm-id`. Run `pip install -r requirements.txt` and then `bash run_all.sh` (about 4–5 hours on a 2-core CPU). The README explains each step.

References

[1] X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, T. B. Hashimoto. *AlpacaEval: An Automatic Evaluator of Instruction-following Models.* GitHub repository `tatsu-lab/alpaca_eval`, 2023.

[2] Y. Kim. Convolutional Neural Networks for Sentence Classification. *EMNLP*, 2014.

[3] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8), 1997.

[4] I. Loshchilov and F. Hutter. Decoupled Weight Decay Regularization. *ICLR*, 2019.

[5] M. A. Covington and J. D. McFall. Cutting the Gordian Knot: The Moving-Average Type-Token Ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2), 2010.